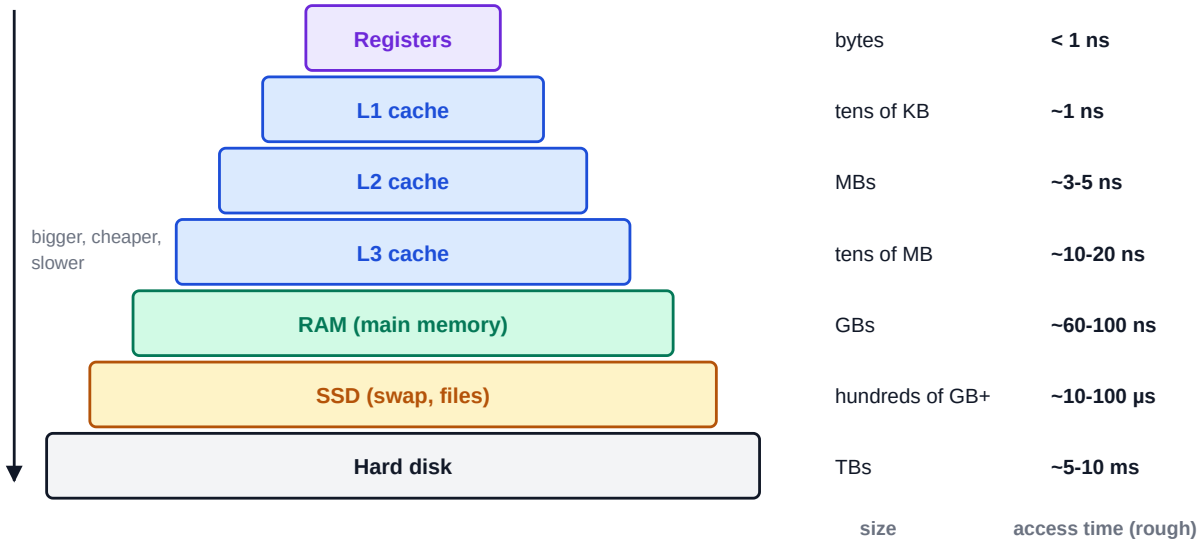
OPERATING SYSTEMS & SECURITY · MODULE 3

# Memory Management & Paging

Lab cheat sheet: how the OS gives out memory, how virtual addresses become physical ones, what happens on a page fault, who gets evicted when RAM is full, and how memory is protected. Commands marked "tested" were run on a Linux box. Example numbers are made up.

## 1. Memory basics and the hierarchy




Everything a program runs (code and data) must be in RAM, or the CPU can't use it. Caches keep copies of hot data closer to the core. Times are rough orders of magnitude.

| Unit | Size |
|---|---|
| bit | one 0 or 1 |
| byte | 8 bits (each byte in RAM has its own address) |
| word | the CPU's natural chunk: 8 bytes on a 64-bit CPU (Intel docs still call 2 bytes a "word") |
| page | 4,096 bytes (4 KB) on most Linux systems |

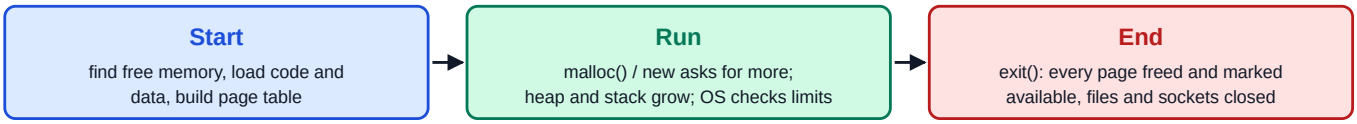
### CHECK YOUR MACHINE

| | |
|---|---|
| `lscpu \| grep -i cache` | L1, L2, L3 sizes on your CPU |
| `free -h` | RAM and swap: total, used, available |
| `cat /proc/meminfo` | every memory counter the kernel keeps |
| `getconf PAGESIZE` | page size in bytes (4096) |
| `sudo dmidecode -t memory` | the physical RAM sticks |

## 2. From code to a process in memory




The compiler turns source into machine code for one CPU architecture. The loader maps that file into a new address space when it runs.




The OS hands out memory when a process starts, manages it while it runs, and takes all of it back at exit.

| | Static | Dynamic |
|---|---|---|
| Libraries | copied into the program at build time | loaded at run time (`.so` on Linux, `.dll` on Windows) |
| File size | big (addr test: 785 KB) | small (addr test: 16 KB) |
| Memory | every program carries its own copy | one copy in RAM shared by every program |
| Security | must rebuild to get a library fix | patch the library once; risk: library hijacking (`LD_PRELOAD`, writable lib paths) |

**TRY IT (TESTED)**

| | |
|---|---|
| `gcc addr.c -o addr` | dynamic (the default) |
| `gcc -static addr.c -o addr_static` | static |
| `ldd ./addr` | shared libraries it will load |
| `strace -e trace=brk,mmap ./addr` | watch it ask the kernel for memory |
| `ps -o pid,vsz,rss,cmd -p <PID>` | VSZ = virtual size, RSS = actually in RAM |
| `valgrind --leak-check=full ./prog` | find memory leaks and bad frees |

## 3. Logical vs physical addresses

| Address | Who uses it | Think of it as |
|---|---|---|
| **Logical / virtual** | the program | a seat number on your ticket |
| **Physical** | the RAM chips | the actual seat in the stadium |
| **Relative / offset** | inside a structure | "5 seats from the aisle" |

The **MMU** (memory management unit, a part of the CPU) translates every logical address into a physical one. The **OS** sets up the tables it uses. Two processes can both use address `0x4000` and still land in different physical RAM: that's the isolation.

**Address space per process:** 4 GB on 32-bit. On x86-64 Linux, 128 TB for the program (256 TB total with 4-level paging).

| | |
|---|---|
| `getconf PAGESIZE` | page size used for translation |
| `setarch -R <cmd>` | run once with ASLR turned off |

**SEE IT (TESTED)**

```
// addr.c: where do things live?
#include <stdio.h>
#include <stdlib.h>
int global = 1;                                 // data segment
int main(void) {
    int local = 2;                              // stack
    int *heap = malloc(sizeof(int));            // heap
    printf("code  (main)   %p\n", (void *)main);
    printf("data  (global) %p\n", (void *)&global);
    printf("heap  (malloc) %p\n", (void *)heap);
    printf("stack (local)  %p\n", (void *)&local);
    free(heap);
    return 0;
}
```

```
$ gcc addr.c -o addr && ./addr
code  (main)   0x5583058831a9     <- low: text
data  (global) 0x558305886010
heap  (malloc) 0x55830a65e2a0     <- grows up
stack (local)  0x7ffebd497b7c     <- high: stack

$ ./addr            # run again: every address moved (ASLR)
$ setarch -R ./addr # ASLR off for this run: same addresses every time
```

| | |
|---|---|
| `cat /proc/<PID>/maps` | every region of a process: address range, permissions, file |

## 4. Virtual memory and swapping

**Virtual memory** gives every process its own private, continuous-looking address space, translated to real RAM by the MMU. **Swapping** is one feature built on top of it: when RAM is full, inactive pages go to disk and come back when needed. It lets you run more than fits in RAM, but disk is far slower, so heavy swapping makes everything crawl.

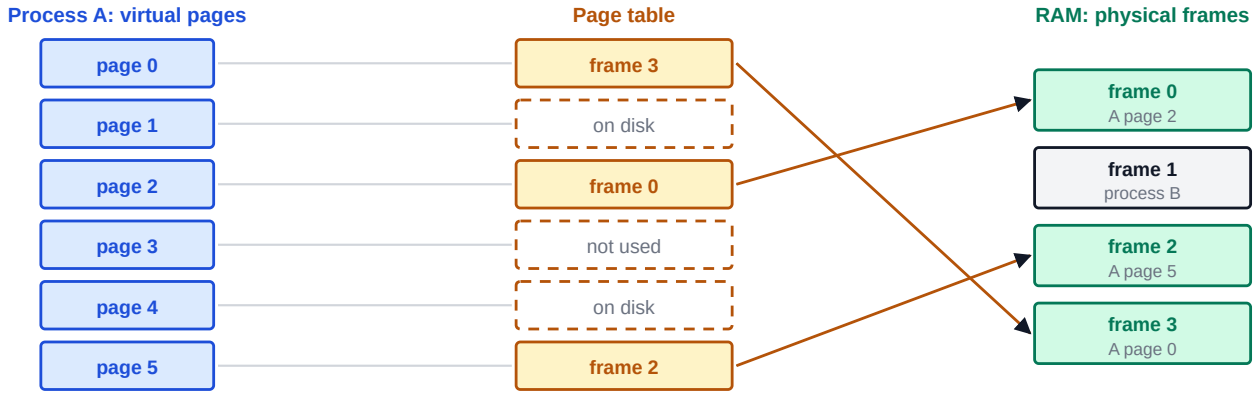
### WATCH IT

| | |
|---|---|
| `free -h` | `available` is what matters, not `free` |
| `swapon --show` | swap partitions or files in use |
| `vmstat 1 5` | `si` / `so` = KB swapped in/out per second (should be near 0) |
| `cat /proc/sys/vm/swappiness` | 0-200: how eagerly Linux swaps (default 60) |
| `sudo sysctl vm.swappiness=10` | swap less (until reboot) |

### WHEN MEMORY RUNS OUT

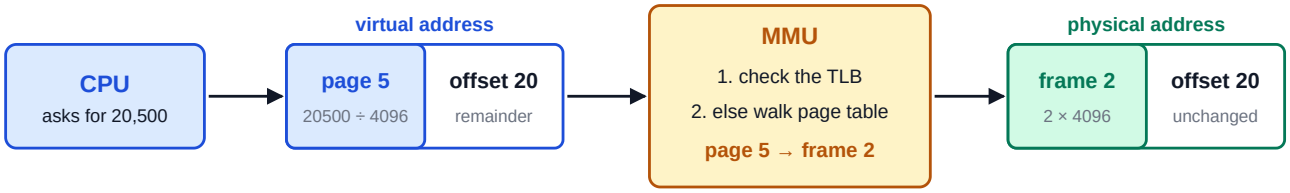
| | |
|---|---|
| `sudo dmesg \| grep -i 'killed process'` | the OOM killer ended something when memory ran out |
| `journalctl -k \| grep -i oom` | same, from the journal |
| `ps -eo pid,user,rss,cmd --sort=-rss \| head` | biggest RAM users |
| `grep -i huge /proc/meminfo` | huge pages (2 MB) status |

## 5. Paging: pages, frames, page tables




Pages and frames are the same size. A page can sit in any free frame, so a process doesn't need one big continuous block of RAM.

Virtual memory is cut into fixed-size **pages**, RAM into same-size **frames**. Each process's **page table** says which frame holds each page, or that it's on disk.




Physical address = 2 × 4096 + 20 = 8,212. In hex: 0x5014 → page 0x5, offset 0x014 (with 4 KB pages, the last 3 hex digits are the offset).

Worked example with 4 KB pages. Page number = address ÷ page size. Offset = the remainder. Only the page number changes; the offset carries over.

| Page table entry bit | Means |
|---|---|
| Present / valid | page is in RAM (0 = page fault) |
| Frame number | where in RAM |
| Read / write | can it be written? |
| User / supervisor | can user mode touch it, or kernel only? |
| NX (no execute) | block running code from this page |
| Accessed / dirty | used recently? changed since loaded? |

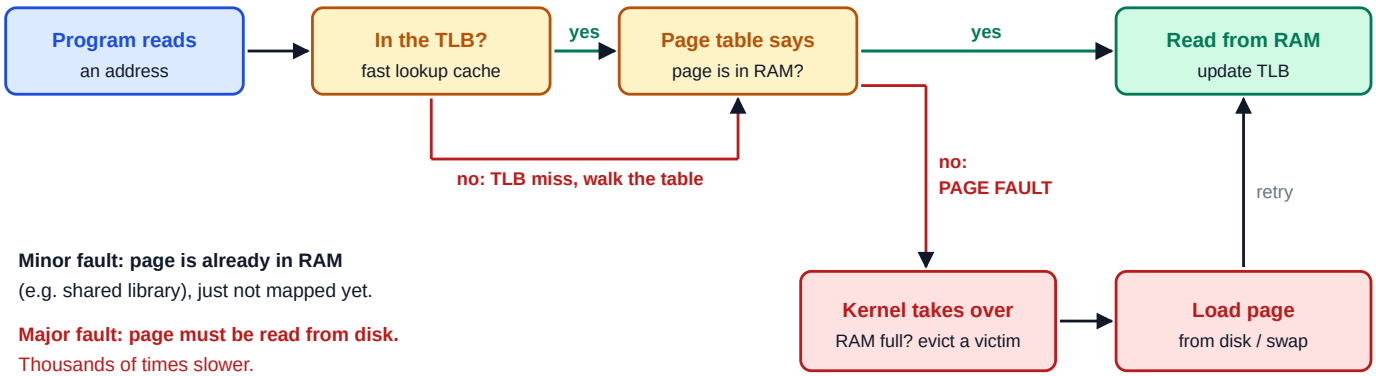
### PRACTICE THE MATH

Page size 4,096. Address 13,000: page = 13,000 ÷ 4,096 = **3**, offset = 13,000 − 12,288 = **712**. If page 3 is in frame 7: physical = 7 × 4,096 + 712 = **29,384**.

**Bigger pages** (2 MB huge pages) mean a smaller page table and fewer TLB misses, but more internal fragmentation.

| | |
|---|---|
| `python3 -c 'print(divmod(13000, 4096))'` | check your page/offset math: (3, 712) |

## 6. The TLB and page faults




**Minor fault: page is already in RAM**
(e.g. shared library), just not mapped yet.

**Major fault: page must be read from disk.**
Thousands of times slower.

The **TLB** (translation lookaside buffer) is a small cache of recent page → frame lookups inside the MMU. Most lookups hit it. A **page fault** is the hardware telling the kernel "this page isn't in RAM".

### COUNT FAULTS

| | |
|---|---|
| `ps -o pid,min_flt,maj_flt,cmd -p <PID>` | minor and major faults so far |
| `/usr/bin/time -v <cmd>` | faults for one run (install the `time` package) |
| `perf stat -e page-faults,dTLB-load-misses <cmd>` | faults and TLB misses (install `perf`) |

**Thrashing:** too many processes for the RAM, so the system spends its time moving pages in and out instead of running programs. Signs: high `si` / `so` in `vmstat`, high `wa` in `top`, disk light always on. Fix: add RAM, run fewer things, or find the leak.

**Avoiding faults:** enough RAM, good replacement (keep hot pages), locality (programs that use nearby data), and prefetching.

## 7. Page replacement: who gets evicted?

RAM has room for 3 frames. The program asks for pages in this order: `7 0 1 2 0 3 0 4 2 3 0 3`. Red cells are faults that loaded a page.

**FIFO: evict the page that has been in RAM the longest**

| page asked | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| frame 0 | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 |
| frame 1 | | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 2 | 2 | 2 | 2 |
| frame 2 | | | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 3 | 3 | 3 |
| result | fault | fault | fault | fault | hit | fault | fault | fault | fault | fault | fault | hit |

**10** faults

**LRU: evict the page that hasn't been used for the longest time**

| page asked | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| frame 0 | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 |
| frame 1 | | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 3 | 3 |
| frame 2 | | | 1 | 1 | 1 | 3 | 3 | 3 | 2 | 2 | 2 | 2 |
| result | fault | fault | fault | fault | hit | fault | hit | fault | fault | fault | fault | hit |

**9** faults

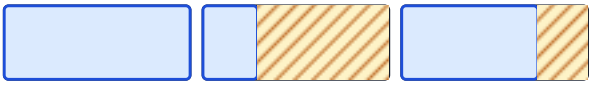
| Algorithm | Evicts | Faults here |
|---|---|---|
| FIFO | oldest page loaded | 10 |
| LRU | least recently used | 9 |
| Optimal | page not needed for the longest time (needs the future, so only a benchmark) | 7 |
| Clock / second chance | like FIFO, but skips pages with the accessed bit set | close to LRU, cheap |

**Belady's anomaly:** with FIFO, giving a process *more* frames can sometimes cause *more* faults. LRU and Optimal never do that.

**Linux** uses an approximation of LRU (active and inactive page lists; multi-generational LRU in newer kernels), because true LRU is too expensive to track on every memory access.

## 8. Fragmentation and compaction

**Internal: waste INSIDE a block**



needs 4,096 B | needs 1,200 B | needs 3,000 B

Each 4 KB page is handed out whole. Unused tails are wasted.

**Paging avoids external fragmentation**
(any page fits any frame) but has some internal fragmentation.

**External: free space too scattered**




**6 KB free in total, but a 5 KB request fails:**
no single hole is big enough.

**Compaction moves A, B, C together:**




Two kinds of waste. Paging trades external fragmentation for a little internal fragmentation.

| Placement strategy | Picks |
|---|---|
| First fit | first hole big enough (fast) |

**CHECK IT**

| | |
|---|---|
| `cat /proc/buddyinfo` | free RAM blocks by size: few big blocks = fragmented |

| | |
|---|---|
| Best fit | smallest hole that fits (leaves tiny unusable slivers) |
| Worst fit | largest hole (leaves big leftovers) |

| | |
|---|---|
| `echo 1 \| sudo tee /proc/sys/vm/compact_memory` | ask the kernel to compact RAM now |
| `filefrag -v <file>` | how many pieces a file is split into on disk |
| `sudo e4defrag <file or dir>` | defragment on ext4 (hard disks) |
| `sudo fstrim -av` | SSDs: run TRIM instead of defragmenting |

## 9. Segmentation

**Segment table (one per process)**

| Seg | Name | Base | Limit | Rights |
|---|---|---|---|---|
| 0 | code | 1000 | 800 | r-x |
| 1 | data | 3000 | 500 | rw- |
| **2** | **stack** | **6000** | **1000** | **rw-** |
| 3 | heap | 8000 | 2000 | rw- |

**Address = (segment, offset)**

(2, 300): 300 < limit 1000 → OK
**physical = base 6000 + 300 = 6300**

(2, 1200): 1200 ≥ limit 1000
**→ trap → "Segmentation fault"**

Writing to segment 0 (r-x) also traps.

**Segmentation** splits memory by purpose (code, data, stack, heap) into variable-size segments. Each has a base, a limit and access rights, checked on every access.

| | Paging | Segmentation |
|---|---|---|
| Block size | fixed (4 KB pages) | variable, one per purpose |
| Address | page number + offset | segment number + offset |
| Fragmentation | internal (unused end of a page) | external (holes between segments) |
| Today | what Linux, Windows and macOS rely on | mostly turned off on x86-64 (flat model); the name survives in "segmentation fault" |

Today a **segfault** usually means a program touched a page it isn't allowed to (unmapped, or wrong permission). `dmesg | grep segfault` shows which program and address.

## 10. Memory protection and security

| Protection | Stops |
|---|---|
| Separate page tables | one process reading another's memory |
| User / supervisor bit | programs touching kernel memory |
| NX / DEP | running injected code from stack or heap |
| Read-only text pages | programs rewriting their own code |
| ASLR | attackers guessing where things are |
| Guard pages | stack growing silently into other memory |

**LAB CHECKS**

| | |
|---|---|
| `cat /proc/<PID>/maps` | permission column: `r-xp` code, `rw-p` data; `rwx` is a red flag |
| `grep -o -w nx /proc/cpuinfo \| head -1` | CPU supports NX |
| `cat /proc/sys/kernel/randomize_va_space` | 2 = full ASLR |
| `grep . /sys/devices/system/cpu/vulnerabilities/*` | Meltdown, Spectre, etc.: mitigated or vulnerable |

| | |
|---|---|
| KPTI | Meltdown-style leaks of kernel memory |

| Bug | Risk |
|---|---|
| Memory leak | slow exhaustion, then OOM kill (DoS) |
| Use-after-free / double free | attacker controls freed memory: code execution |
| Heap / stack overflow | overwrite nearby data or pointers |
| Secrets in swap or core dumps | passwords and keys written to disk |

| | |
|---|---|
| `ulimit -c` | core dumps (0 = off) |
| `swapon --show / lsblk -f` | is swap on an encrypted volume? |
| `cat /proc/sys/kernel/kptr_restrict` | hides kernel addresses from normal users (1 or 2) |

## 11. Common mix-ups

- **Virtual memory isn't just "using the disk as RAM."** It's the private address space plus translation. Swap is one thing it makes possible.
- **Swapping doesn't make things faster.** Disk is thousands of times slower than RAM; it keeps programs alive, not quick.
- **A "word"** depends on the CPU. On 64-bit it's 8 bytes, though Intel naming still calls 2 bytes a word.
- **L2 cache** is usually private to each core on modern CPUs. L3 is the shared one.
- **CPU registers** are the fastest storage, but they don't have memory addresses. Physical addresses point at RAM (and device memory).
- **Page vs frame:** page = a chunk of *virtual* memory; frame = a same-size slot of *physical* RAM.
- **Bigger pages aren't free:** smaller page tables and fewer TLB misses, but more internal fragmentation.
- **Don't defragment an SSD.** It gains nothing and adds wear; TRIM is what SSDs need.
- **Compaction** works on RAM. **Defragmentation** works on files on disk.